

REPRESENTATION LEARNING · EPISODE 01

Demystifying Vanilla Autoencoders

From first principles to PyTorch.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

The Hook

I gave a neural network a 784-number input and forced the whole signal through a 2-number slit.

Then I told it to redraw the original input from those two numbers.

The result taught me more about data than any supervised model ever did.

The Intuition

Take a 28×28 handwritten digit (like from the MNIST dataset).

In pixel space, it is represented by 784 numbers. That is a 784-dimensional space.

If you were to pick random coordinates in that 784-dimensional space, you would almost certainly generate pure, TV-static noise. The probability of randomly generating the shape of a "3" is essentially zero.

So where does the data live?

Real data is not random.

The Manifold Hypothesis

The shape of a "3" really lives on a thin, curled surface inside that massive 784-dimensional space.

Real data rarely fills the space; it sits on a low-dimensional folded surface. This is known as the **Manifold Hypothesis**.

To build intelligent systems, we need to mathematically unroll that surface. If we can compress the 784 dimensions down to the intrinsic dimensions of the manifold, we have successfully extracted the "meaning" of the data.



Why Not PCA?

Classical PCA finds straight axes of maximum variance. For data that lies on a flat sheet, that's fine. But real-world manifolds are rarely flat; think of a Swiss Roll where the data curls through space.

Yes, there are non-linear extensions like Kernel PCA. Those can learn curves by implicitly mapping data into a high-dimensional feature space. But they come with trade-offs: you must guess the right kernel function upfront, and the computation doesn't scale gracefully to massive datasets.

Autoencoders take a different path. Instead of a pre-defined mapping, they *learn* the non-linear bending directly from the data.

No kernel to guess, no linear assumption baked in—just a flexible neural network that moulds itself around the manifold. Crucially, the bottleneck also yields a generative direction: you can reconstruct, not just project.

The choice isn't "PCA bad, AE good." It's about what you need: a fast linear map (PCA), a kernel-driven curve (kPCA), or a learned, scalable transformation (Autoencoder).

The Core Idea: The Bottleneck

Enter the Autoencoder.

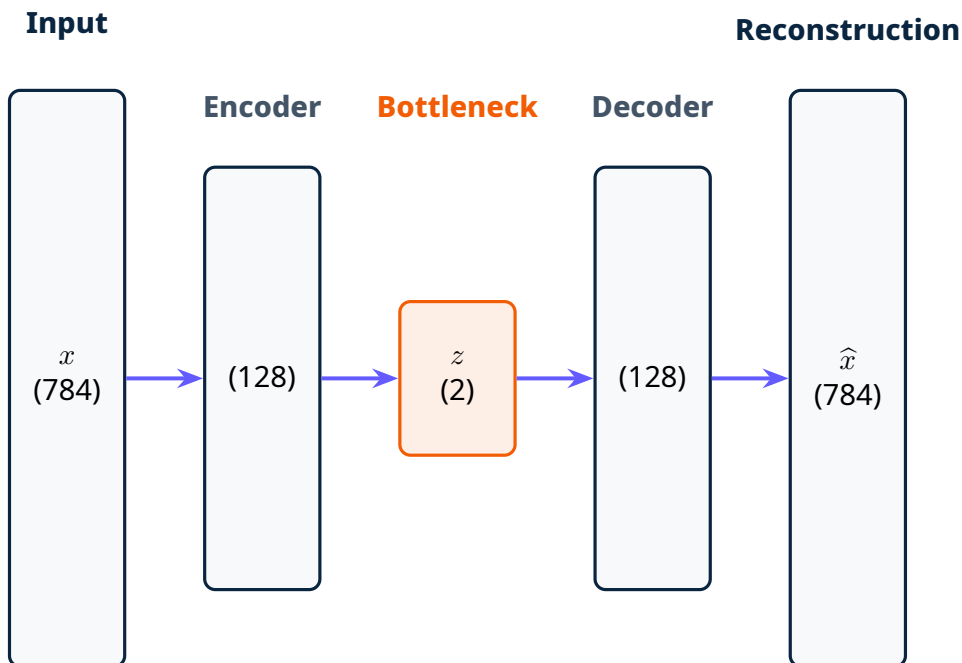
The Autoencoder works by starving the network of space. It is exactly like trying to describe a whole painting using only five words.

Because you are forced through a restrictive bottleneck, you have to throw away the noise and the background details. You are forced to keep only the absolute most essential structural parts of the image to make sure the listener understands.

The network is **forced** to learn structure because it **must** compress to survive the bottleneck.

The Architecture

The architecture looks like this—input on the left, a tiny bottleneck in orange in the middle, and the reconstruction on the right.



The Probabilistic Lens

The hourglass architecture design isn't accidental. It falls out of a simple generative story: we imagine the data came from a hidden low-dimensional code z (the **prior**), then passed through some complex process that produced the high-dimensional x (the **evidence**).

The decoder $p(x|z)$ models how data is generated: given a code z , what image x would we expect?

But in practice we only see the image x . To recover the hidden code, we need the inverse—the **posterior** $p(z|x)$, which tells us which latent codes could explain the observed data.

That posterior is mathematically intractable, so we approximate it with the encoder $q(z|x)$.

That's the whole architecture, in two lines:

- **Encoder** = approximate posterior (compress evidence into a latent code)
- **Decoder** = likelihood/evidence model (reconstruct evidence from the code)

The reconstruction loss is just maximum **likelihood** under this model—we're maximising how well the evidence is explained by the inferred code.

The hourglass shape? Forced by the fact that we're solving an inverse problem with a bottleneck—exactly the same reasoning that gives us the manifold hypothesis.

The First Principle

The math isn't a complex derivation. It's just two non-linear functions and a measurement of failure.

1. The Encoder

Compresses the input down into the tiny latent code:

$$\mathbf{z} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

2. The Decoder

Attempts to expand that tiny code back into the original input:

$$\hat{\mathbf{x}} = \sigma'(\mathbf{W}_2\mathbf{z} + \mathbf{b}_2)$$

3. The Loss

We just measure the pixel-wise difference and backpropagate that error. No labels needed.

$$\mathcal{L} = \text{mean}(\|\mathbf{x} - \hat{\mathbf{x}}\|^2)$$

The Deceptive Simplicity

At first glance, an autoencoder is just learning the identity function: $f(x) = x$. That would be trivial—a network with enough capacity could simply memorise the input and spit it back unchanged, like a photocopier. Completely useless.

The bottleneck changes the game. By forcing the data through a tiny latent code z , we break the identity. The network *can't* just copy; it must discard. It must discover the most salient, compressible features that allow reconstruction—the true signal beneath the noise.

This is why an undercomplete autoencoder (where $p \ll d$; p = the dimensionality of the bottleneck, d = dimensionality of the input data) is not a bug but deliberately designed. It's a forced abstraction engine.

An even more explicit way to prevent identity-copying is the **denoising autoencoder**: instead of feeding x directly, we feed a corrupted version $\tilde{x} = x + \eta$ (where η is noise) and demand the reconstruction of the clean x .

The loss becomes $\mathcal{L} = ||x - D(E(\tilde{x}))||^2$. Here, $E(\cdot)$ = Encoder FxN, $D(\cdot)$ = Decoder FxN. The network can no longer cheat at all; it must truly understand the underlying structure to undo the noise.

This tiny twist— $f(x + \eta) = x$ instead of $f(x) = x$ —turns a copier into a pattern extractor. It's a beautiful, simple principle that makes the whole framework work.

Code: The Model Definition

models/vanilla_ae.py

```
import torch
import torch.nn as nn

class VanillaAutoencoder(nn.Module):
    def __init__(self, input_dim=784, latent_dim=2):
        super().__init__()

        # Compress down to the bottleneck
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim) # Latent Space 'z'
        )

        # Expand back to original dimensions
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid()
        )

    def forward(self, x):
        # No labels. Just x in, x-hat out.
        return self.decoder(self.encoder(x))
```

Code: The Training Loop

Notice how we never provide a 'target' or 'label' vector. The model learns entirely through self-supervision.

train.py

```
import torch.optim as optim

model = VanillaAutoencoder()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss() # L = mean(||x - x_hat||^2)

# Training loop
for epoch in range(num_epochs):
    for batch_features, _ in dataloader:

        # Flatten image to 784 vector
        batch_features = batch_features.view(-1, 784)

        # Forward pass (x -> z -> x_hat)
        outputs = model(batch_features)

        # Calculate reconstruction loss (x vs x_hat)
        loss = criterion(outputs, batch_features)

        # Backpropagate the error
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

Where it Hits Reality

1. Anomaly Detection

Train only on normal transactions. Fraud comes along, the reconstruction error spikes—because the model never learned to compress that weird pattern.

Set a threshold τ and you have a detector that never saw a single label.



Code: Detecting Anomalies in Production

At inference time, detecting an anomaly is as simple as running a forward pass and measuring the loss.

inference.py

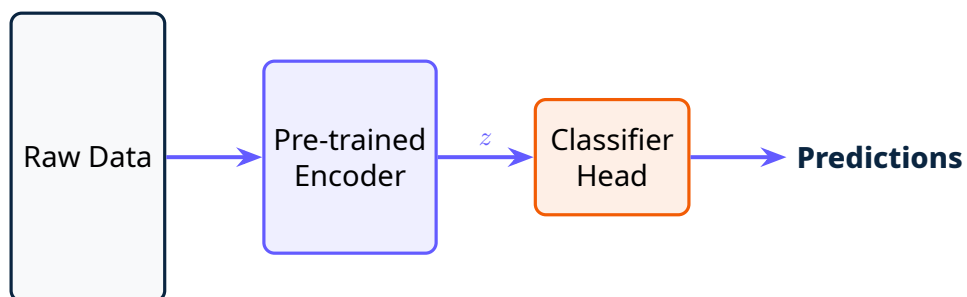
```
def detect_anomaly(transaction_data, model, threshold=0.05):  
    model.eval()  
    with torch.no_grad():  
        # Reconstruct the transaction  
        reconstruction = model(transaction_data)  
  
        # Calculate Mean Squared Error  
        mse = torch.mean((transaction_data - reconstruction)**2)  
  
        # If error exceeds our tuned threshold tau  
        if mse.item() > threshold:  
            return True, mse.item() # Flag as Fraud!  
  
    return False, mse.item() # Normal transaction
```

Unsupervised Pre-training

2. Feature Extraction

Train the encoder on mountains of unlabelled data. Then throw away the decoder and attach a small classifier.

With very few labelled examples, you get a strong feature extractor—it learned what matters without being told.



The Deeper Principle

Compression is intelligence.

Not the file-zipping kind. The kind that looks at a thousand noisy examples and extracts the one rule that explains them all.

A bottleneck doesn't just reduce dimensions—it forces a model to *understand* what can be thrown away and what must stay. That's not just signal processing. That's the seed of abstraction.

Forget the layers. Remember the constraint.

COMING UP NEXT...

Variational Autoencoders: When the Latent Code Becomes a Probability Distribution

Same hourglass, one critical twist: instead of a point, the encoder outputs a distribution.

That tiny change makes the latent space smooth, continuous, and generative—you can walk through it and *create* new data.

Building production-grade ML, one first-principle at a time.

github.com/MrHeaven1y
linkedin.com/in/dibayendu-mukherjee-bb897b267

Dibayendu Mukherjee